TP Hadoop

Analyse des ventes par ville

Rapport technique et preuves d'exécution

Information	
Nom et prénom	NGUYEN Thérèse
Formation	Big Data - RNCP 40573 - Option BC05 Data et IA
Organisme	DIGINAMIC
Formateur	Robin HOTTON
Date de remise	22/09/2026
Version	2

Document de travail

Présentation du rapport

Ce rapport documente la chaîne de traitement mise en œuvre pour analyser les ventes par ville. Il présente le contexte de chaque étape, les commandes et codes exécutés, les retours obtenus, les preuves par capture d'écran et les conclusions tirées. Les résultats sont validés de manière croisée entre MapReduce, HBase, Hive et la visualisation Python.

Table des matières

Présentation du rapport 2
I Objectifs et périmètre 4
II Préparation de l'environnement 5
2.1 Lancement des conteneurs Docker 5
2.2 Ouverture d'un terminal dans le master 5
2.3 Processus du cluster et rôles 5
2.4 Dépôt du fichier de données dans le master 7
2.5 Dépôt et vérification dans HDFS 7
2.6 Contrôle des premières et dernières lignes 8
2.7 Analyse de la source et dictionnaire des données 8
III MapReduce 10
3.1 Q1 Articles commandés par ville 10
3.2 Q2 Articles commandés par ville et année 16
IV HBase 24
4.1 Q3 Stockage des totaux par ville 24
V Visualisation 28
5.1 Lecture de HBase avec HappyBase 28
5.2 Production des livrables 29
5.3 Contrôle de cohérence 30
5.4 Récupération sur le poste 31
VI Hive 32
6.1 Préparation du TSV 32
6.2 Dépôt du TSV dans HDFS 34
6.3 Création de la base et de la table externe 34
6.4 Requête reproduisant Q1 35
6.5 Requête reproduisant Q2 36
6.6 Validation croisée avec MapReduce 36
6.7 Analyse du plan avec EXPLAIN 36
VIII Synthèse et validation croisée 38
8.1 Cohérence des résultats 38
8.2 Matrice de contrôle des composants 38
8.3 Conclusion 38

I Objectifs et périmètre

L'objectif du TP est d'exploiter un fichier de ventes dans un environnement Hadoop, puis de comparer plusieurs modes de calcul, de stockage, de requêtage et de visualisation.

Question	Objectif	Technologie principale
Q1	Identifier les villes qui commandent le plus d'articles	MapReduce
Q2	Identifier les couples ville et année les plus importants	MapReduce
Q3	Retrouver le total d'articles d'une ville	HBase et HappyBase
Q4	Reproduire les classements en SQL	Hive
Q5	Automatiser l'ingestion du mini CSV	NiFi

Données utilisées :
dataw_fr03_mini_1000.csv
dataw_fr03.csv

II Préparation de l'environnement

Contexte et objectif

Mise en route du cluster et préparation de l'accès aux données

2.1 Lancement des conteneurs Docker

Commandes ou code

Commande exécutée dans le dossier Hadoop

```
docker compose up -d
```

Retour obtenu

4 containers sont lancés : hadoop-master / hadoop-slave1 / hadoop-slave2 /hadoop-nifi

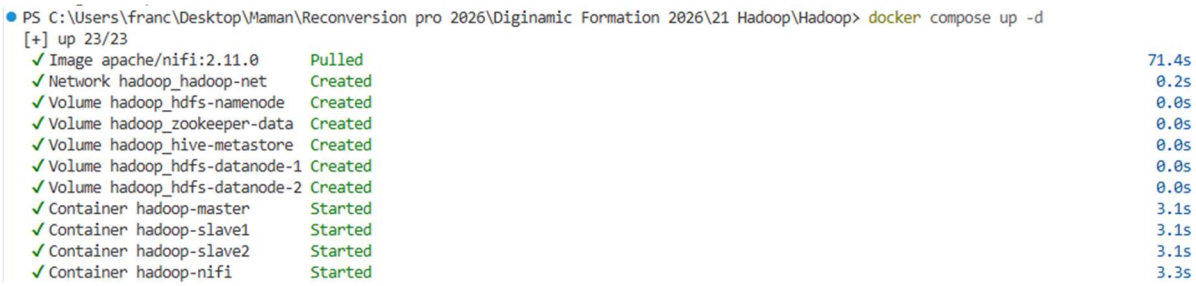


Figure 1 - Conteneurs Docker démarrés

2.2 Ouverture d'un terminal dans le master

Commandes ou code

Commande exécutée dans le dossier Hadoop

```
docker exec -it hadoop-master bash

root@hadoop-master:/home# ls
```

Retour obtenu

Le terminal du conteneur hadoop-master est ouvert et son contenu est affiché.



Figure 2 – Terminal et contenu du conteneur hadoop-master

2.3 Processus du cluster et rôles

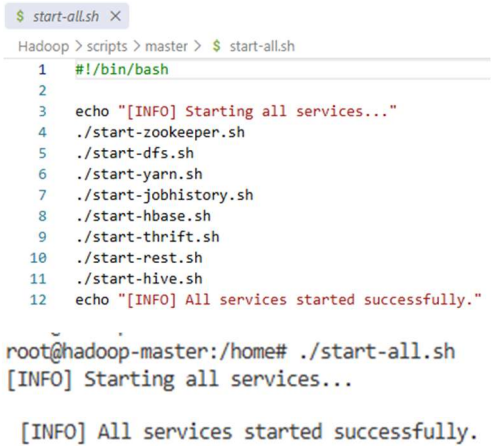
Commandes ou code

Commande exécutée dans le terminal hadoop-master

```
root@hadoop-master:/home# ./start-all.sh
```

Retour obtenu

Le script lance tous les services du cluster dans l'ordre des dépendances : Zookeeper → HDFS → YARN → JobHistory → HBase → Thrift → REST → Hive.



```
$ start-all.sh
Hadoop > scripts > master > $ start-all.sh
#!/bin/bash

echo "[INFO] Starting all services..."
./start-zookeeper.sh
./start-dfs.sh
./start-yarn.sh
./start-jobhistory.sh
./start-hbase.sh
./start-thrift.sh
./start-rest.sh
./start-hive.sh
echo "[INFO] All services started successfully."

root@hadoop-master:/home# ./start-all.sh
[INFO] Starting all services...

[INFO] All services started successfully.
```

Figure 3 – script 'start-all.sh' et démarrage de tous les services

Commandes ou code

Commande exécutée dans le terminal hadoop-master

```
root@hadoop-master:/home# jps
```

Retour obtenu

La commande jps (Java VM Process Status) permet d'identifier les processus Java actifs dans le conteneur.



```
root@hadoop-master:/home# jps
865 JobHistoryServer
1731 RESTServer
53 QuorumPeerMain
711 ResourceManager
263 NameNode
3048 JarBootstrapMain
1512 HMaster
4281 Jps
2045 RunJar
446 SecondaryNameNode
1631 ThriftServer
```

Figure 4 - Résultat de la commande jps

Rôle des processus

Processus	Composant	Rôle
QuorumPeerMain	ZooKeeper	Coordination des services distribués, notamment HBase
NameNode	HDFS	Gestion des métadonnées, fichiers et blocs HDFS
SecondaryNameNode	HDFS	Création périodique de checkpoints des métadonnées
ResourceManager	YARN	R épartion des ressources et planification des applications
JobHistoryServer	MapReduce	Conservation de l'historique des jobs terminés
HMaster	HBase (NoSQL)	Administration des tables

Processus	Composant	Rôle
ThriftServer	HBase	Accès à HBase pour HappyBase et d'autres clients
RESTServer	HBase	Exposition de HBase au moyen d'une API REST
RunJar	Java ou Hive (SGBDR)	Lancement d'une application empaquetée dans un fichier JAR
Jps	JDK	Affichage ponctuel des processus Java actifs
JarBootstrapMain	Java	Processus générique de démarrage d'une application JAR

2.4 Dépôt du fichier de données dans le master

Commandes ou code

Commande exécutée dans le dossier Hadoop

```
docker cp dataw_fr03.csv hadoop-master:/home/
```

Retour obtenu

Le fichier de données dataw_fr03.csv est copié dans le master.

```
PS C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\Hadoop>
docker cp "..\TP\dataw_fr03.csv" hadoop-master:/home/
Successfully copied 26.4MB (transferred 26.4MB) to hadoop-master:/home/
```

Figure 5 - Copie du CSV dans le conteneur master

2.5 Dépôt et vérification dans HDFS

Commandes ou code

Commande exécutée dans le terminal hadoop-master

```
root@hadoop-master:/home# hdfs dfs -put dataw_fr03.csv input

root@hadoop-master:/home# hdfs dfs -ls input
```

Retour obtenu

Le fichier est stocké dans HDFS. Nous le vérifions via le terminal et également via l'interface web (http://localhost:9870).

```
root@hadoop-master:/home#  hdfs dfs -ls input
Found 2 items
-rw-r--r--   1 root supergroup     156547 2026-09-15 14:39 input/Spotify_Most_Streamed_Songs.csv
-rw-r--r--   1 root supergroup   26397166 2026-09-18 17:52 input/dataw_fr03.csv
```

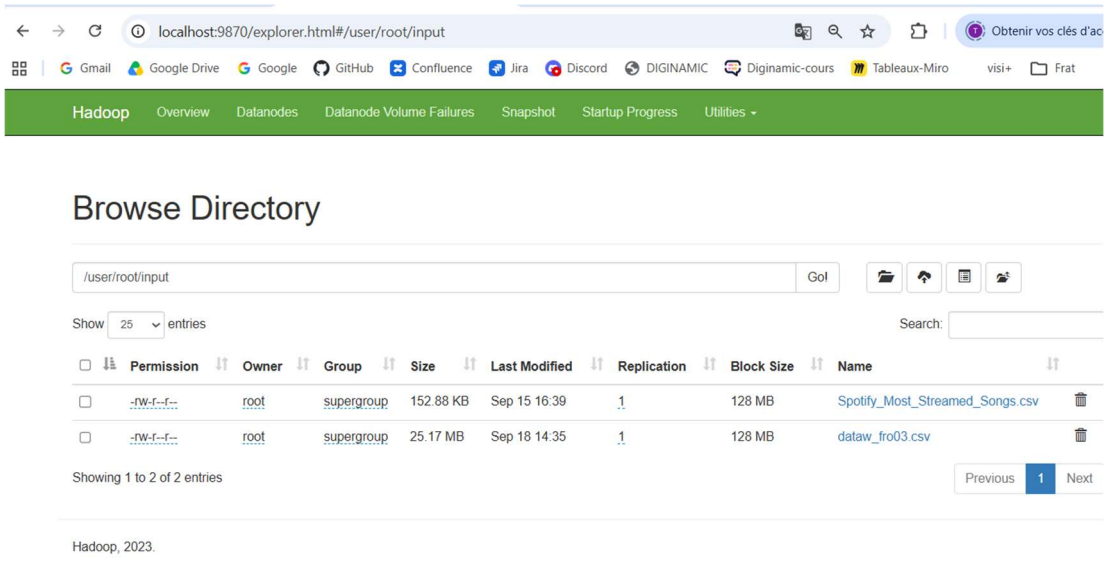


Figure 6 - Fichier de données 'data_fr03.csv' présent dans HDFS

2.6 Contrôle des premières et dernières lignes

Commandes ou code

Commande exécutée dans le terminal hadoop-master

```
root@hadoop-master:/home# hdfs dfs -cat input/dataw_fr03.csv | head -5
root@hadoop-master:/home# hdfs dfs -cat input/dataw_fr03.csv | tail -5
```

Retour obtenu

Affichage des 5 premières et 5 dernières lignes du fichier 'dataw_fr03.csv'.

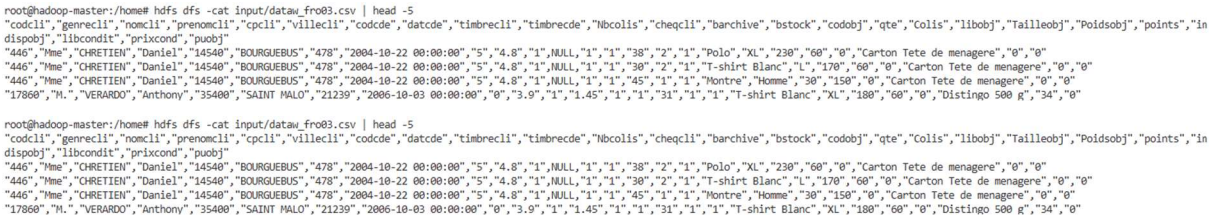


Figure 7 - Contrôle du contenu depuis HDFS

Le fichier de données a bien été copié.

2.7 Analyse de la source et dictionnaire des données

La source de données est représentée par 2 fichiers : dataw_fro03.csv et dataw_fro03_mini_1000.csv

Le 2e fichier est un extrait des 1000 premières lignes du 1er fichier.

Les fichiers représentent les commandes de clients.

Le fichier de données complet comporte 135278 lignes de données (dont la 1ère correspond à l'en-tête des colonnes) réparties sur 25 colonnes distinctes, séparées par une virgule.

Une ligne représente un article associé à une commande et contient des informations sur le client, la commande, l'article et son conditionnement. Une commande peut comporter plusieurs lignes.

Le dictionnaire des données décrit le sens, le type logique et l'utilisation de chaque champ.

Colonne	Type logique	Description	Exemple
codcli	Entier	Identifiant du client	1234
genrecli	Texte	Civilité du client	Mme, M., M. & Mme ou Melle
nomcli	Texte Capital	Nom du client	DUPONT
prenomcli	Texte	Prénom du client	Jean
cpcli	Texte	Code postal du client	75000
villecli	Texte Capital	Ville du client	PARIS
codcde	Entier	Identifiant de la commande	1234
datcde	Date et heure	Date de la commande	2005-07-21 00 :00 :00
timbrecli	Décimal	Montant d'affranchissement associé au client	1.2
timbrecde	Décimal	Montant d'affranchissement de la commande	3.4
Nbcolis	Entier	Nombre de colis de la commande	5
cheqcli	Décimal	Montant associé au règlement par chèque	1.23 ou NULL
barchive	Booléen	Indicateur d'archivage	1
bstock	Booléen	Indicateur lié au stock	1
codobj	Entier	Identifiant de l'article	78
qte	Entier	Quantité commandée pour l'article	3
Colis	Entier	Numéro ou indicateur du colis	1
libobj	Texte	Libellé de l'article	Montre, Louche ou Casquette
Tailleobj	Texte	Taille ou variante de l'article	Femme, XL, Bordeaux ou NULL
Poidsobj	Entier	Poids de l'article	123
points	Entier	Nombre de points associé à l'article	45
indispobj	Booléen	Indicateur d'indisponibilité	0
libcondit	Texte	Libellé du conditionnement	Distingo 100 g, enveloppe polo, carton peluche
prixcond	Entier	Prix du conditionnement	178
puobj	Décimal	Prix unitaire de l'article	0 (pas d'autre valeur semble-t-il)

Ce dictionnaire de données est à valider avec le métier.

Pour les calculs du TP, les colonnes principales sont villecli, datcde et qte. Le nombre d'articles correspond à la somme de qte et non au simple nombre de lignes.

III MapReduce

Cette partie met en œuvre Hadoop Streaming avec des scripts Python. Chaque traitement est d'abord testé localement, puis exécuté sur le cluster.

3.1 Q1 Articles commandés par ville

3.1.1 Code Mapper

Code 'mapper.py'

mapper.py

TP > III_MapReduce_Q1 > mapper.py > ...

```python
import csv
import sys

csv_reader = csv.reader(sys.stdin, delimiter=",")

for row in csv_reader:

    # Ignorer les lignes incomplètes et l'en-tête
    if len(row) < 16 or row[0] == "codcli":
        continue

    ville = row[5].strip()  # colonne villecli
    quantite_str = row[15].strip()  # colonne qte

    try:
        quantite = int(quantite_str)
    except ValueError:
        continue

    if ville:
        print(f"{ville}\t{quantite}")
```

3.1.2 Code Reducer

Code 'reducer.py'

reducer.py

TP > III_MapReduce_Q1 > reducer.py > ...

```python
import sys

current_city = None
total_quantity = 0

for line in sys.stdin:
    line = line.strip()
    parts = line.split("\t", 1)

    if len(parts) < 2:
        continue

    city, quantity_str = parts

    try:
        quantity = int(quantity_str)
    except ValueError:
        continue

    if city == current_city:
        total_quantity += quantity

    else:
        # Afficher le résultat de la ville précédente
        if current_city is not None:
            print(f"{current_city}\t{total_quantity}")

        # Initialiser la nouvelle ville
        current_city = city
        total_quantity = quantity

# Ne pas oublier la dernière ville
if current_city is not None:
    print(f"{current_city}\t{total_quantity}")
```

3.1.3 Test local

Commande exécutée dans le dossier du TP

```
cat .\dataw_fr03_mini_1000.csv -head 10 | python mapper.py | sort | python reducer.py
```

```
PS C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\TP\III_MapReduce_Q1>
cat .\dataw_fro03_mini_1000.csv -head 10 | python mapper.py | sort | python reducer.py
ALENCON 3
BOURGUEBUS      5
ECQUETOT        1
LILLEBONNE      4
MESSEI  3
SAINT MALO      1
```

Figure 8 - Résultat du test local de Q1

On obtient un extrait des villes et la quantité d'articles commandés (séparé par une tabulation).

Sur la totalité du fichier de données (restreint à 1000 lignes pour optimiser l'espace disque) :

```
PS C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\TP\III_MapReduce_Q1>
 Get-Content .\dataw_fro03_mini_1000.csv |
>>     python mapper.py |
>>     Sort-Object |
>>     python reducer.py |
>>     ForEach-Object {
>>         $parties = $_ -split "`t"
>>
>>         [PSCustomObject]@{
>>             Ville = $parties[0]
>>             Total = [int]$parties[1]
>>         }
>>     } |
>>     Sort-Object Total -Descending |
>>     Select-Object -First 5

Ville              Total
-----              -----
LE MANS               30
MAMERS                18
DOMFRONT EN POIRAIE   16
LAVAL                 15
DOUDEVILLE            15
```

3.1.4 Exécution sur le cluster

Script 'job.sh'

job.sh

TP > III_MapReduce_Q1 > $ job.sh

```
# job.sh – script d'automatisation du job MapReduce.
# Le job type écrit sur stdout (pas HBase).

INPUT_FILE="dataw_fro03.csv"
OUTPUT_FILE="TP-Q1"

# Répertoire d'entrée : le CSV source complet hébergé par le TP
hdfs dfs -mkdir -p input
hdfs dfs -put -f $INPUT_FILE input

# Le répertoire de sortie ne doit pas exister : on le vide avant chaque run
hdfs dfs -rm -r -f output/$OUTPUT_FILE
hdfs dfs -mkdir -p output

# Lancement du job Hadoop avec les scripts mapper et reducer
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
-file mapper.py -mapper "python3 mapper.py" \
-file reducer.py -reducer "python3 reducer.py" \
-input input/$INPUT_FILE \
-output output/$OUTPUT_FILE

# Afficher les 5 villes ayant commandé le plus d'articles
hdfs dfs -cat output/$OUTPUT_FILE/part-* \
| sort -k2,2nr \
| head -n 5
```

Commande exécutée dans le dossier du TP (copie des fichiers dans hadoop-master)

```
docker cp mapper.py hadoop-master:/home/

docker cp reducer.py hadoop-master:/home/

docker cp job.sh hadoop-master:/home/
```

Après vérification des fichiers copiés, on lance le script et on vérifie les résultats du job.

Commande exécutée dans le terminal hadoop-master

```
root@hadoop-master:/home# ls -l mapper.py reducer.py
root@hadoop-master:/home# ls -l job.sh
root@hadoop-master:/home# ./job.sh
root@hadoop-master:/home# hdfs dfs -ls output
root@hadoop-master:/home# hdfs dfs -ls output/TP-Q1
```

```
PS C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\TP\III_MapReduce_Q1>
 docker cp mapper.py hadoop-master:/home/
Successfully copied 468B (transferred 2.05kB) to hadoop-master:/home/
PS C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\TP\III_MapReduce_Q1>
 docker cp reducer.py hadoop-master:/home/
Successfully copied 764B (transferred 2.56kB) to hadoop-master:/home/
PS C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\TP\III_MapReduce_Q1>
 docker cp job.sh hadoop-master:/home/
Successfully copied 837B (transferred 2.56kB) to hadoop-master:/home/
PS C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\TP\III_MapReduce_Q1>
```

```
root@hadoop-master:/home# ls -l mapper.py
-rwxr-xr-x 1 root root 468 Sep 18 15:59 mapper.py
root@hadoop-master:/home# ls -l reducer.py
-rwxr-xr-x 1 root root 764 Sep 18 15:59 reducer.py
root@hadoop-master:/home# ls -l job.sh
-rwxr-xr-x 1 root root 837 Sep 18 17:44 job.sh
```

```
root@hadoop-master:/home# ./job.sh
2026-09-18 17:52:59,250 WARN streaming.StreamJob: -file option is deprecated, please use generic option -file
s instead.
packageJobJar: [mapper.py, reducer.py, /tmp/hadoop-unjar3424350182845315717/] [] /tmp/streamjob10686781503253
691590.jar tmpDir=null
2026-09-18 17:53:00,619 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at hado
op-master/172.30.0.10:8032
2026-09-18 17:53:00,870 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at hado
op-master/172.30.0.10:8032
2026-09-18 17:53:01,456 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-ya
rn/staging/root/.staging/job_1789629044281_0001
2026-09-18 17:53:02,121 INFO mapred.FileInputFormat: Total input files to process : 1
2026-09-18 17:53:02,276 INFO mapreduce.JobSubmitter: number of splits:2
2026-09-18 17:53:02,801 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789629044281_0001
2026-09-18 17:53:02,802 INFO mapreduce.JobSubmitter: Executing with tokens: []
```

[...]

```
2026-09-18 17:53:42,706 INFO mapreduce.Job:  map 100% reduce 0%
2026-09-18 17:53:49,792 INFO mapreduce.Job:  map 100% reduce 100%
2026-09-18 17:53:49,819 INFO mapreduce.Job: Job job_1789629044281_0001 completed successfully
2026-09-18 17:53:49,879 INFO mapreduce.Job: Counters: 54
```

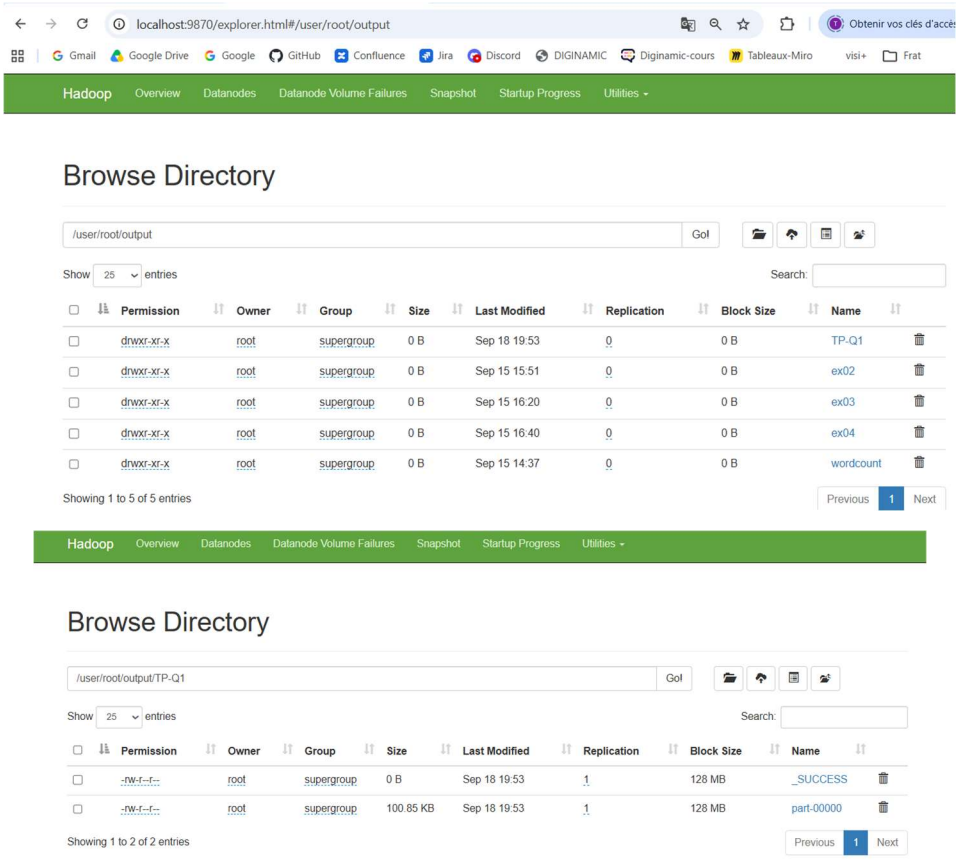
[...]

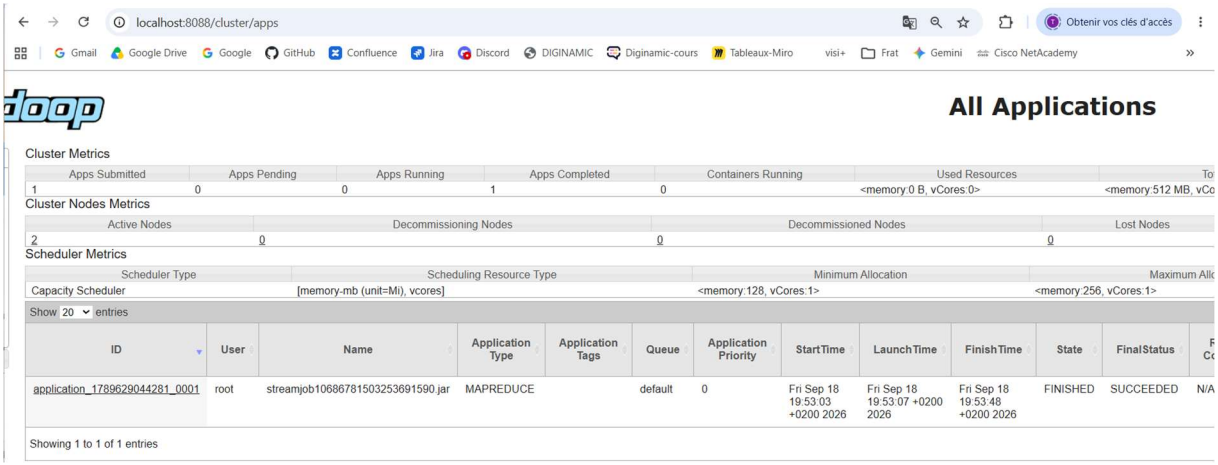
```
        File Input Format Counters
                Bytes Read=26436855
        File Output Format Counters
                Bytes Written=103271
2026-09-18 17:53:49,880 INFO streaming.StreamJob: Output directory: output/TP-Q1
CAEN    2056
FLERS   1480
LAVAL   1422
ALENCON 1166
ARGENTAN        938
```

Figure 9a - Exécution réussie du job MapReduce Q1

```
root@hadoop-master:/home#  hdfs dfs -ls "output"
Found 5 items
drwxr-xr-x   - root supergroup          0 2026-09-18 17:53 output/TP-Q1
```

```
root@hadoop-master:/home#  hdfs dfs -ls "output/TP-Q1"
Found 2 items
-rw-r--r--   1 root supergroup          0 2026-09-18 17:53 output/TP-Q1/_SUCCESS
-rw-r--r--   1 root supergroup     103271 2026-09-18 17:53 output/TP-Q1/part-00000
```





(job terminé avec succès)

Figure 9b – Vérification de l'exécution réussie du job MapReduce Q1

On peut constater la réussite du job, la création du dossier TP-Q1 et des fichiers _SUCCESS et part-00000 via l'interface web ou via les lignes de commande HDFS.

3.1.5 Résultats

Cinq premières lignes du fichier part

Commande exécutée dans le terminal hadoop-master

```
root@hadoop-master:/home# hdfs dfs -cat output/TP-Q1/part-00000 | head -5
ABBEVILLE       29
ABILLY  6
ABLAIN ST NAZAIRE       1
ABLIS   1
ABLON   89
```

Figure 10a -Résultat de MapReduce Q1 (5 premières lignes)

```
root@hadoop-master:/home# hdfs dfs -cat output/TP-Q1/part-00000 | sort -k2,2nr | head -5
CAEN    2056
FLERS   1480
LAVAL   1422
ALENCON 1166
ARGENTAN        938
```

Figure 10b – Résultat de MapReduce Q1 (Top 5)

<u>NOTA : on verra par la suite pourquoi ce résultat est erroné et comment le corriger. (voir Q2)</u>

Top 5 des villes

Rang	Ville	Total articles
1	CAEN	2056
2	FLERS	1480
3	LAVAL	1422
4	ALENCON	1166
5	ARGENTAN	938

3.2 Q2 Articles commandés par ville et année

3.2.1 Code Mapper

Code 'mapper.py'

mapper.py

TP > III_MapReduce_Q2 > mapper.py > ...

```python
import csv
import sys

csv_reader = csv.reader(sys.stdin, delimiter=",")

for row in csv_reader:

    # Ignorer les lignes incomplètes et l'en-tête
    if len(row) < 16 or row[0] == "codcli":
        continue

    ville = row[5].strip()  # colonne villecli
    date_commande = row[7].strip()  # colonne datecde
    quantite_str = row[15].strip()  # colonne qte

    annee = date_commande [:4]

    try:
        quantite = int(quantite_str)
    except ValueError:
        continue

    if ville and len(annee) == 4 and annee.isdigit():
        print(f"{ville}#{annee}\t{quantite}")  # clé composite ville#annee

```

3.2.2 Code Reducer

Code 'reducer.py'

reducer.py

TP > III_MapReduce_Q2 > reducer.py > ...

```python

current_composite_key = None
total_quantity = 0

for line in sys.stdin:
    line = line.strip()
    parts = line.split("\t", 1)

    if len(parts) < 2:
        continue

    composite_key, quantity_str = parts

    try:
        quantity = int(quantity_str)
    except ValueError:
        continue

    if composite_key == current_composite_key:
        total_quantity += quantity

    else:
        # Afficher le résultat de la clé composite ville#anne précédente
        if current_composite_key is not None:
            print(f"{current_composite_key}\t{total_quantity}")

        # Initialiser la clé composite ville#annee
        current_composite_key = composite_key
        total_quantity = quantity

# Ne pas oublier la dernière clé composite
if current_composite_key is not None:
    print(f"{current_composite_key}\t{total_quantity}")

```

3.2.3 Test local

Commande exécutée dans le dossier du TP

```
cat .\dataw_fr03_mini_1000.csv -head 10 | python mapper.py | sort | python reducer.py
```

```
PS C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\TP\III_MapReduce_Q2>
 cat .\dataw_fro03_mini_1000.csv -head 10 | python mapper.py | sort | python reducer.py
ALENCON#2004    3
BOURGUEBUS#2004 5
ECQUETOT#2004   1
LILLEBONNE#2004 4
MESSEI#2004     3
SAINT MALO#2006 1
```

Figure 11 - Résultat du test local de Q2

On obtient un extrait des (ville#annee) et la quantité d'articles commandés (séparé par une tabulation).

Sur la totalité du fichier de données (restreint à 1000 lignes pour optimiser l'espace disque) :

```
PS C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\TP\III_MapReduce_Q2>
 Get-Content .\dataw_fro03_mini_1000.csv |
>>     python mapper.py |
>>     Sort-Object |
>>     python reducer.py |
>>     ForEach-Object {
>>         $parties = $_ -split "`t"
>>
>>         [PSCustomObject]@{
>>             VilleAnnee = $parties[0]
>>             Total      = [int]$parties[1]
>>         }
>>     } |
>>     Sort-Object Total -Descending |
>>     Select-Object -First 5

VilleAnnee               Total
----------               -----
LE MANS#2006                30
MAMERS#2006                 18
DOMFRONT EN POIRAIE#2006    16
DOUDEVILLE#2006             15
LAVAL#2006                  15
```

On obtient le TOP5 des (ville#année) ayant le plus vendu d'articles.

3.2.3 Exécution sur le cluster

Script 'job.sh'

$ job.sh ×

TP > III_MapReduce_Q2 > $ job.sh

```
# job.sh — script d'automatisation du job MapReduce.
# Le job type écrit sur stdout (pas HBase).

INPUT_FILE="dataw_fro03.csv"
OUTPUT_FILE="TP-Q2"

# Répertoire d'entrée : le CSV source complet hébergé par le TP
hdfs dfs -mkdir -p input
hdfs dfs -put -f $INPUT_FILE input

# Le répertoire de sortie ne doit pas exister : on le vide avant chaque run
hdfs dfs -rm -r -f output/$OUTPUT_FILE
hdfs dfs -mkdir -p output

# Lancement du job Hadoop avec les scripts mapper et reducer
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
-file mapper.py -mapper "python3 mapper.py" \
-file reducer.py -reducer "python3 reducer.py" \
-input input/$INPUT_FILE \
-output output/$OUTPUT_FILE

# Afficher les 5 (ville#année) ayant commandé le plus d'articles
hdfs dfs -cat output/$OUTPUT_FILE/part-* \
| sort -k2,2nr \
| head -5
```

On copie les fichiers mapper.py, reducer.py et job.sh dans le master.

Commande exécutée dans le dossier du TP

```
docker cp mapper.py hadoop-master:/home/

docker cp reducer.py hadoop-master:/home/

docker cp job.sh hadoop-master:/home/
```

Après vérification des fichiers copiés, on lance le script et on vérifie les résultats du job.

Commande exécutée dans le terminal hadoop-master

```
root@hadoop-master:/home# ls -l mapper.py reducer.py

root@hadoop-master:/home# ls -l job.sh

root@hadoop-master:/home# ./job.sh

root@hadoop-master:/home# hdfs dfs -ls output

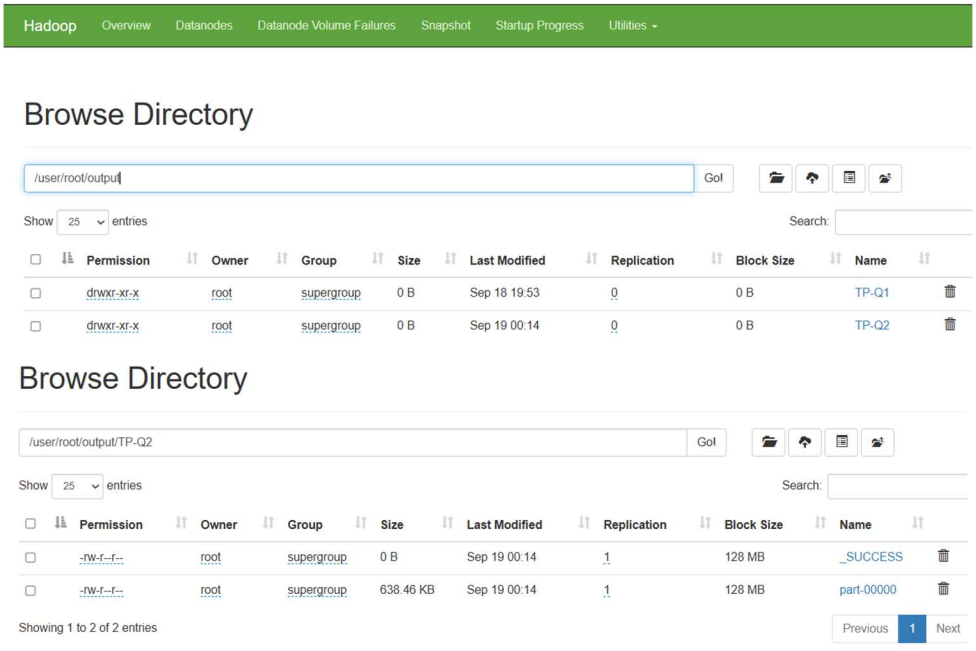
root@hadoop-master:/home# hdfs dfs -ls output/TP-Q2
```

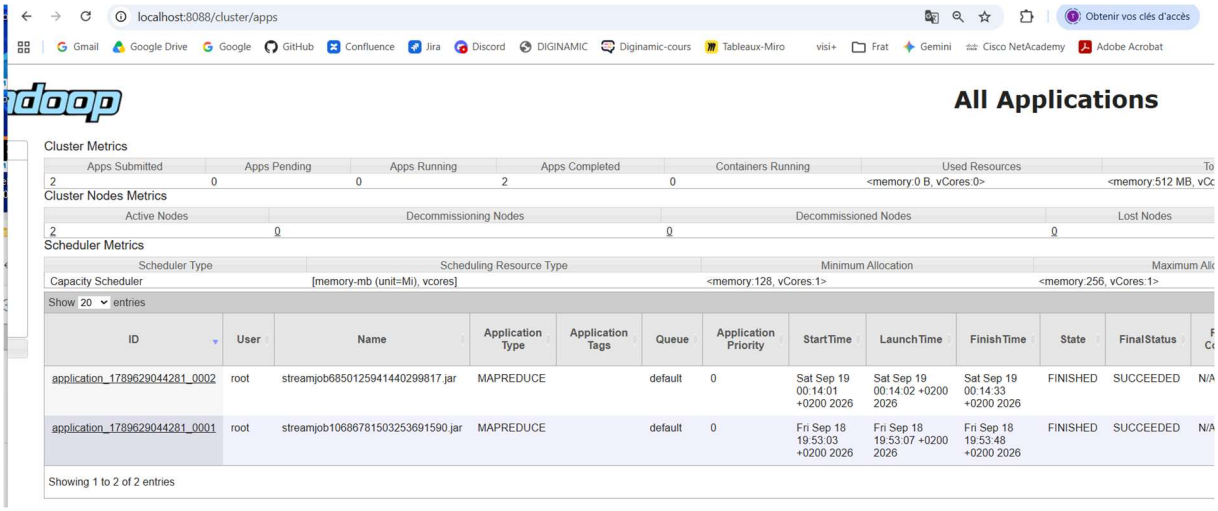
```
2026-09-18 22:14:22,221 INFO mapreduce.Job:  map 50% reduce 0%
2026-09-18 22:14:28,279 INFO mapreduce.Job:  map 100% reduce 0%
2026-09-18 22:14:35,355 INFO mapreduce.Job:  map 100% reduce 100%
2026-09-18 22:14:35,370 INFO mapreduce.Job: Job job_1789629044281_0002 completed successfully
2026-09-18 22:14:35,437 INFO mapreduce.Job: Counters: 55
        File System Counters
```

[...]

```
        File Output Format Counters
                Bytes Written=653779
2026-09-18 22:14:35,437 INFO streaming.StreamJob: Output directory: output/TP-Q2
PRAHA 616#2007  1
CAEN#2008       195
CAEN#2010       185
CAEN#2009       175
CAEN#2016       170
```

```
root@hadoop-master:/home#  hdfs dfs -ls output
Found 6 items
drwxr-xr-x   - root supergroup          0 2026-09-18 17:53 output/TP-Q1
drwxr-xr-x   - root supergroup          0 2026-09-18 22:14 output/TP-Q2
drwxr-xr-x   - root supergroup          0 2026-09-15 13:51 output/ex02
drwxr-xr-x   - root supergroup          0 2026-09-15 14:20 output/ex03
drwxr-xr-x   - root supergroup          0 2026-09-15 14:40 output/ex04
drwxr-xr-x   - root supergroup          0 2026-09-15 12:37 output/wordcount
root@hadoop-master:/home#  hdfs dfs -ls output/TP-Q2
Found 2 items
-rw-r--r--   1 root supergroup          0 2026-09-18 22:14 output/TP-Q2/_SUCCESS
-rw-r--r--   1 root supergroup     653779 2026-09-18 22:14 output/TP-Q2/part-00000
```





(job terminé avec succès)

Figure 12 - Exécution réussie du job MapReduce Q2

On peut constater la réussite du job, la création du dossier TP-Q2 et des fichiers _SUCCESS et part-00000 via l'interface web ou via les lignes de commande HDFS

3.2.5 Résultats

Cinq premières lignes du fichier part

Commande exécutée dans le terminal hadoop-master

```
root@hadoop-master:/home# hdfs dfs -cat output/TP-Q2/part-00000 | head -5
ABBEVILLE#2008  2
ABBEVILLE#2010  3
ABBEVILLE#2012  10
ABBEVILLE#2015  7
ABBEVILLE#2019  6
```

Figure 13a -Résultat de MapReduce Q2 (5 premières lignes)

```
root@hadoop-master:/home# hdfs dfs -cat output/TP-Q2/part-00000 | sort -k2,2nr | head -5
PRAHA 616#2007  1
CAEN#2008       195
CAEN#2010       185
CAEN#2009       175
CAEN#2016       170
```

Figure 13b – Résultat de MapReduce Q2 (Top 5)

On constate un problème au niveau du TOP5 : PRAHA 616#2007 est 1er avec 1 article vendu. Il y a un problème de tri au niveau de la ville : le séparateur à prendre en compte est une tabulation et non un espace.

Correction apportée à 'job.sh' (ligne 24 à corriger : | sort -k2,2nr \ ) :

$ job.sh ×

TP > III_MapReduce_Q2 > $ job.sh

```
22  # Afficher les 5 (ville#année) ayant commandé le plus d'articles
23  hdfs dfs -cat output/$OUTPUT_FILE/part-* \
24  | sort -t $'\t' -k2,2nr \
25  | head -5
26
```

La commande « sort -t $'\t' -k2,2nr » va trier le fichier issu du MaReduce avec :

- le séparateur = tabulation (afin qu'une ville comportant 1 espace ou 1 nombre reste une clé unique)
- le critère de tri = deuxième colonne
- n = tri numérique
- r = reverse (ordre décroissant)

En relançant le job corrigé :

```
                Bytes Written=653779
2026-09-18 22:50:16,529 INFO streaming.StreamJob: Output directory: output/TP-Q2
BEAUFORT EN VALLEE#2012 305
LE MANS#2010    305
LE MANS#2006    267
LE MANS#2011    262
LE MANS#2015    258
```

On retrouve bien le TOP5 avec des villes en plusieurs mots (caractère ' ' (espace) entre les mots) qui sont prises en compte : la commande sort ne prend plus en compte l'espace entre les mots pour le séparateur entre clé composite et valeur du total articles.

```
root@hadoop-master:/home# hdfs dfs -cat output/TP-Q2/part-00000 | sort -t $'\t' -k2,2nr | head -5
BEAUFORT EN VALLEE#2012 305
LE MANS#2010    305
LE MANS#2006    267
LE MANS#2011    262
LE MANS#2015    258
```

Figure 13c – Résultat de MapReduce Q2 (Top 5) après correction

Le TOP20 présente bien des villes en 1 ou plusieurs mots :

```
root@hadoop-master:/home# hdfs dfs -cat output/TP-Q2/part-00000 | sort -t $'\t' -k2,2nr | head -20
BEAUFORT EN VALLEE#2012 305
LE MANS#2010    305
LE MANS#2006    267
LE MANS#2011    262
LE MANS#2015    258
LE MANS#2008    250
LE MANS#2007    196
CAEN#2008       195
CAEN#2010       185
LE MANS#2009    185
LE MANS#2012    182
LE MANS#2016    180
CAEN#2009       175
CHERBOURG EN COTENTIN#2014      174
LE MANS#2014    171
CAEN#2016       170
LE MANS#2013    167
LAVAL#2006      162
LE MANS#2018    159
CAEN#2006       157
```

Rang	Ville et année	Total articles
1	BEAUFORT EN VALLEE#2012	305
2	LE MANS#2010	305
3	LE MANS#2006	267
4	LE MANS#2011	262
5	LE MANS#2015	258

La correction du 'sort' a également été apportée à la question Q1 :

$ job.sh ...\III_MapReduce_Q2 × $ job.sh ...\III_MapReduce_Q1 ×

TP > III_MapReduce_Q1 > $ job.sh

```
22  # Afficher les 5 villes ayant commandé le plus d'articles
23  hdfs dfs -cat output/$OUTPUT_FILE/part-* \
24  | sort -t $'\t' -k2,2nr \
25  | head -5
26
```

Le job a été relancé :

```
        File Output Format Counters
                Bytes Written=103271
2026-09-18 23:01:17,491 INFO streaming.StreamJob: Output directory: output/TP-Q1
LE MANS 3019
CAEN    2056
FLERS   1480
LAVAL   1422
VIRE NORMANDIE  1405
```

Le TOP5 de la question Q1 est corrigé (il intègre à présent les villes en 1 ou plusieurs mots) :

```
        File Output Format Counters
                Bytes Written=103271
2026-09-18 23:01:17,491 INFO streaming.StreamJob: Output directory: output/TP-Q1
LE MANS 3019
CAEN    2056
FLERS   1480
LAVAL   1422
VIRE NORMANDIE  1405
```

Top 5 des villes (après correction de 'job.sh')

Rang	Ville	Total articles
1	LE MANS	3019
2	CAEN	2056
3	FLERS	1480
4	LAVAL	1422
5	VIRE NORMANDIE	1405

Conclusion :

Les traitements MapReduce sont fonctionnels aussi bien en local que dans le cluster. Les résultats sont stockés dans les fichiers part-*

Nous avons pu calculer le top5 des villes qui commandent le plus, puis le top5 des couples ville#année.

IV HBase

4.1 Q3 Stockage des totaux par ville

4.1.1 Préparation de la table

Au préalable, on vérifie la connection à happybase :

```
root@hadoop-master:/home# python3 -c "import happybase; c=happybase.Connection('hadoop-master',9090); print(c.tables()); c.close()"
[b'bibliotheque', b'chrono', b'streams_artist', b'streams_year']
```

Commande exécutée dans le terminal hadoop-master

```
root@hadoop-master:/home# python3 - <<'PY'
import happybase

connection = happybase.Connection("hadoop-master", 9090)

if b"articles_ville" not in connection.tables():
    connection.create_table(
        "articles_ville",
        {"stat": dict()}
    )
    print("Table articles_ville créée")
else:
    print("Table articles_ville existe déjà")

connection.close()
PY
Table articles_ville créée
```

La table vide ‘articles_ville’ est créée dans HBase avec une connection Happybase.

```
hbase:007:0> list
TABLE
articles_ville
bibliotheque
chrono
streams_artist
streams_year
5 row(s)
Took 0.3908 seconds
=> ["articles_ville", "bibliotheque", "chrono", "streams_artist", "streams_year"]
```

Figure 14 - Table HBase ‘articles_ville’ préparée pour Q3

4.1.2 Reducer HappyBase

Code 'reducer.py' écrivant dans HBase

reducer.py 1

TP > IV_HBase_Q3 > reducer.py > ...

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import sys
import happybase

HBASE_HOST = "hadoop-master"
HBASE_PORT = 9090
TABLE_NAME = "articles_ville"

current_city = None
total_quantity = 0
rows = []

################
# AGGREGATION #
################

for line in sys.stdin:
    line = line.strip()
    parts = line.split("\t", 1)

    if len(parts) < 2:
        continue

    city, quantity_str = parts

    try:
        quantity = int(quantity_str)
    except ValueError:
        continue

    if city == current_city:
        total_quantity += quantity

    else:
        # Afficher le résultat de la ville précédente
        if current_city is not None:
            rows.append((current_city, total_quantity))

        # Initialiser la nouvelle ville
        current_city = city
        total_quantity = quantity

# Ne pas oublier la dernière ville
if current_city is not None:
    rows.append((current_city, total_quantity))

#########
# HBASE #
#########
connection = happybase.Connection(HBASE_HOST, HBASE_PORT)
table = connection.table(TABLE_NAME)

# Insertions groupées dans HBase
with table.batch(batch_size=100) as batch:
    for city, quantity in rows:
        batch.put(
            city.encode("utf-8"),
            {
                b"stat:total_quantity":
                    str(quantity).encode("utf-8")
            }
        )

connection.close()

# Les messages de contrôle vont dans stderr et non stdout
print(
    f"{len(rows)} lignes insérées dans {TABLE_NAME}",
    file=sys.stderr
)
```

On copie le fichier reducer.py dans le master.

Commande exécutée dans le dossier du TP

```
docker cp reducer.py hadoop-master:/home/
```

4.1.3 Exécution du job

Commande exécutée dans le terminal hadoop-master

```
root@hadoop-master:/home# hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-D mapreduce.job.reduces=1 \
-file mapper.py \
-mapper "python3 mapper.py" \
-file reducer.py \
-reducer "python3 reducer.py" \
-input input/dataw_fro03.csv \
-output output/TP-Q3
```

```
26-09-20 21:26:22,543 INFO mapreduce.Job:  map 100% reduce 0%
26-09-20 21:26:31,639 INFO mapreduce.Job:  map 100% reduce 100%
26-09-20 21:26:31,662 INFO mapreduce.Job: Job job_1789895948523_0005 completed successfully
26-09-20 21:26:31,764 INFO mapreduce.Job: Counters: 54
      File System Counters

                                    [...]

                Shuffle Errors
                        BAD_ID=0
                        CONNECTION=0
                        IO_ERROR=0
                        WRONG_LENGTH=0
                        WRONG_MAP=0
                        WRONG_REDUCE=0
                File Input Format Counters
                        Bytes Read=26436855
                File Output Format Counters
                        Bytes Written=0
        2026-09-20 21:26:31.765 INFO streaming.StreamJob: Output directory: output/TP-03
```

Figure 15 - Exécution du job MapReduce vers HBase

4.1.4 Vérification avec hbase shell

Commande exécutée dans le terminal hadoop-master

```
root@hadoop-master:/home# hbase shell
```

Le hbase shell se lance.

On vérifie que le format est celui attendu (extrait des 10 premières lignes) :

```
hbase:009:0> scan 'articles_ville', { LIMIT => 10 }
ROW                                     COLUMN+CELL
 ABBEVILLE                              column=stat:total_quantity, timestamp=2026-09-20T21:26:28.390, value=29
 ABILLY                                 column=stat:total_quantity, timestamp=2026-09-20T21:26:28.390, value=6
 ABLAIN ST NAZAIRE                      column=stat:total_quantity, timestamp=2026-09-20T21:26:28.390, value=1
 ABLIS                                  column=stat:total_quantity, timestamp=2026-09-20T21:26:28.390, value=1
 ABLON                                  column=stat:total_quantity, timestamp=2026-09-20T21:26:28.390, value=89
 ABLON SUR SEINE                        column=stat:total_quantity, timestamp=2026-09-20T21:26:28.390, value=1
 ABONDANT                               column=stat:total_quantity, timestamp=2026-09-20T21:26:28.390, value=12
 ACHERES                                column=stat:total_quantity, timestamp=2026-09-20T21:26:28.390, value=9
 ACHERES LA FORET                       column=stat:total_quantity, timestamp=2026-09-20T21:26:28.390, value=3
 ACIGNE                                 column=stat:total_quantity, timestamp=2026-09-20T21:26:28.390, value=25
10 row(s)
Took 0.1327 seconds
```

Commandes de contrôle dans hbase shell

```
count 'articles_ville'
get 'articles_ville', 'LE MANS'
```

Contrôle	Résultat obtenu	Interprétation
Nombre de lignes	6255	Il y a 6255 villes référencées.
Valeur de LE MANS	3019	Valeur identique que Q1 (MapReduce)

```
hbase:008:0> count 'articles_ville'
Current count: 1000, row: CHARENTILLY
Current count: 2000, row: GRAND FORT PHILIPPE
Current count: 3000, row: LESCONIL
Current count: 4000, row: PLEUMEUR GAUTIER
Current count: 5000, row: SAINT SULPICE SUR RISLE
Current count: 6000, row: VAUDRIMESNIL
6255 row(s)
Took 0.2977 seconds
=> 6255
hbase:002:0> get 'articles_ville', 'LE MANS'
COLUMN                          CELL
 stat:total_quantity            timestamp=2026-09-20T21:26:29.378, value=3019
1 row(s)
Took 0.2707 seconds
```

Figure 16 - Count et valeur LE MANS dans HBase

Vérification du TOP5 :

```
hbase:005:0> scan 'articles_ville', {FILTER => "SingleColumnValueFilter('stat','total_quantity', =, 'binary:3019')"}
ROW                             COLUMN+CELL
 LE MANS                        column=stat:total_quantity, timestamp=2026-09-20T21:26:29.378, value=3019
1 row(s)
Took 0.0499 seconds
hbase:006:0> scan 'articles_ville', {FILTER => "SingleColumnValueFilter('stat','total_quantity', =, 'binary:2056')"}
ROW                             COLUMN+CELL
 CAEN                           column=stat:total_quantity, timestamp=2026-09-20T21:26:28.632, value=2056
1 row(s)
Took 0.0512 seconds
hbase:007:0> scan 'articles_ville', {FILTER => "SingleColumnValueFilter('stat','total_quantity', =, 'binary:1480')"}
ROW                             COLUMN+CELL
 FLERS                          column=stat:total_quantity, timestamp=2026-09-20T21:26:29.010, value=1480
1 row(s)
Took 0.0478 seconds
hbase:008:0> scan 'articles_ville', {FILTER => "SingleColumnValueFilter('stat','total_quantity', =, 'binary:1422')"}
ROW                             COLUMN+CELL
 LAVAL                          column=stat:total_quantity, timestamp=2026-09-20T21:26:29.301, value=1422
1 row(s)
Took 0.0418 seconds
hbase:009:0> scan 'articles_ville', {FILTER => "SingleColumnValueFilter('stat','total_quantity', =, 'binary:1405')"}
ROW                             COLUMN+CELL
 VIRE NORMANDIE                 column=stat:total_quantity, timestamp=2026-09-20T21:26:29.939, value=1405
1 row(s)
Took 0.0673 seconds
```

Conclusion :

Les résultats agrégés par ville (rowkey=ville) sont enregistrés dans HBase via Happybase qui communique avec le ThriftServer.

Les commandes exécutées dans hbase shell permettent de retrouver le même résultat qu'avec MapReduce.

V Visualisation

5.1 Lecture de HBase avec HappyBase

Script Python de visualisation 'graphique_city.py'

graphique_city.py 3

TP > V_Visualisation > graphique_city.py > ...

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import happybase

import matplotlib
matplotlib.use("Agg")

import matplotlib.pyplot as plt

HBASE_HOST = "hadoop-master"
HBASE_PORT = 9090
TABLE_NAME = "articles_ville"
COLUMN = b"stat:total_quantity"
TOP_N = 10

connection = happybase.Connection(
    HBASE_HOST,
    HBASE_PORT
)

table = connection.table(TABLE_NAME)

cities = []

# Lecture de toutes les lignes de la table HBase
for row_key, data in table.scan():

    if COLUMN not in data:
        continue

    city = row_key.decode("utf-8")
    total_quantity = int(data[COLUMN].decode("utf-8"))

    cities.append((city, total_quantity))

connection.close()

if not cities:
    raise RuntimeError(
        "La table articles_ville ne contient aucune donnée."
    )

# Classement décroissant
cities.sort(
    key=lambda item: item[1],
    reverse=True
)
```

```
49
50  top_cities = cities[:TOP_N]
51
52  # Création du fichier texte
53  with open("top10.txt", "w", encoding="utf-8") as fichier:
54      fichier.write("Rang\tVille\tTotal_articles\n")
55
56      for ranking, (city, total_quantity) in enumerate(top_cities, start=1):
57          fichier.write(f"{ranking}\t{city}\t{total_quantity}\n")
58  print(f"TXT généré : top10.txt")
59
60  # Inverser pour placer la meilleure ville en haut du classement
61  top_cities.reverse()
62
63  # Création du graphique
64  names = [city for city, total in top_cities]
65  articles_quantity = [total for city, total in top_cities]
66
67  plt.figure(figsize=(12, 9))
68
69  plt.barh(
70      names,
71      articles_quantity,
72      color="steelblue"
73  )
74
75  plt.xlabel("Nombre total d'articles vendus")
76  plt.ylabel("Ville")
77  plt.title(
78      f"Top {TOP_N} des villes par nombre d'articles vendus"
79  )
80
81  plt.grid(
82      axis="x",
83      linestyle="--",
84      alpha=0.4
85  )
86
87  plt.tight_layout()
88  plt.savefig("resultat.pdf")
89
90  print(
91      f"PDF généré : resultat.pdf "
92      f"({len(cities)} villes analysées)"
93  )
```

5.2 Production des livrables

Exécution du script (après copie du fichier sur hadoop-master)

```
python3 graphique_city.py
```

Livrable	Contenu	Contrôle effectué
resultat.pdf	Graphique du top 10	OK, idem top10.txt
top10.txt	Classement texte de contrôle	OK, idem graphique

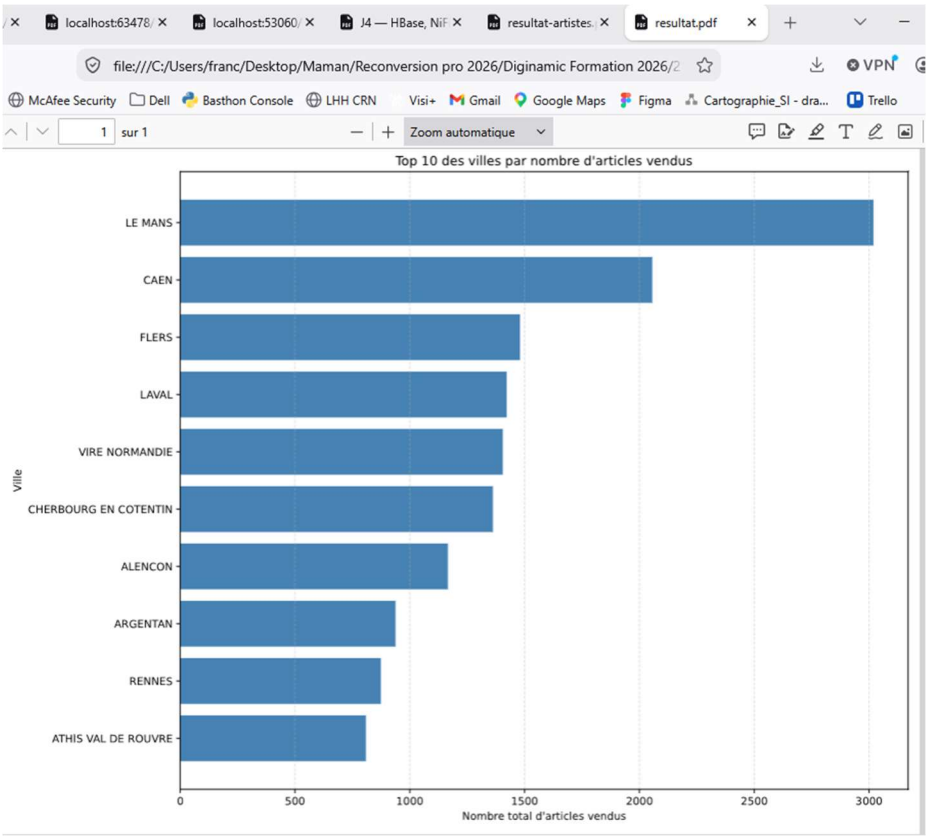


Figure 17 - Graphique Matplotlib du top 10 des villes

Contenu de top10.txt

```
root@hadoop-master:/home# python3 graphique_city.py
TXT généré : top10.txt
PDF généré : resultat.pdf (6255 villes analysées)
root@hadoop-master:/home# cat top10.txt
Rang    Ville   Total_articles
1       LE MANS 3019
2       CAEN    2056
3       FLERS   1480
4       LAVAL   1422
5       VIRE NORMANDIE  1405
6       CHERBOURG EN COTENTIN   1362
7       ALENCON 1166
8       ARGENTAN        938
9       RENNES  874
10      ATHIS VAL DE ROUVRE     809
```

5.3 Contrôle de cohérence

Rang	Ville	MapReduce	HBase	Graphique	Conforme
1	LE MANS	✓	✓	✓	Oui / ~~Non~~
2	CAEN	✓	✓	✓	Oui / ~~Non~~
3	FLERS	✓	✓	✓	Oui / ~~Non~~
4	LAVAL	✓	✓	✓	Oui / ~~Non~~
5	VIRE NORMANDIE	✓	✓	✓	Oui / ~~Non~~

Conclusion :

Le script Python indépendant qui lit HBase donne les même résultats que précédemment.

5.4 Récupération sur le poste

Copie HDFS vers le conteneur hadoop-master

```
root@hadoop-master:/home# mkdir -p livrables/TP-Q1
root@hadoop-master:/home# mkdir -p livrables/TP-Q2
root@hadoop-master:/home# hdfs dfs -get output/TP-Q1/part-* livrables/TP-Q1/
root@hadoop-master:/home# hdfs dfs -get output/TP-Q2/part-* livrables/TP-Q2/
```

Copie du conteneur hadoop-master vers Windows

```
docker cp hadoop-master:/home/livrables .\livrables

docker cp hadoop-master:/home/resultat.pdf .\livrables\

docker cp hadoop-master:/home/top10.txt .\livrables\
```

```
root@hadoop-master:/home# ls -al livrables
total 0
drwxr-xr-x 1 root root 512 Sep 20 22:55 .
drwxrwxrwx 1 root root 512 Sep 20 22:55 ..
drwxr-xr-x 1 root root 512 Sep 20 22:56 TP-Q1
drwxr-xr-x 1 root root 512 Sep 20 22:56 TP-Q2
```

```
root@hadoop-master:/home# ls -al livrables/TP-Q1
total 104
drwxr-xr-x 1 root root    512 Sep 20 22:56 .
drwxr-xr-x 1 root root    512 Sep 20 22:55 ..
-rw-r--r-- 1 root root 103271 Sep 20 22:56 part-00000
root@hadoop-master:/home# ls -al livrables/TP-Q2
total 640
drwxr-xr-x 1 root root    512 Sep 20 22:56 .
drwxr-xr-x 1 root root    512 Sep 20 22:55 ..
-rw-r--r-- 1 root root 653779 Sep 20 22:56 part-00000
```

```
root@hadoop-master:/home# head -5 livrables/TP-Q1/part-00000
ABBEVILLE       29
ABILLY  6
ABLAIN ST NAZAIRE       1
ABLIS   1
ABLON   89
root@hadoop-master:/home# head -5 livrables/TP-Q2/part-00000
ABBEVILLE#2008  2
ABBEVILLE#2010  3
ABBEVILLE#2012  10
ABBEVILLE#2015  7
ABBEVILLE#2019  6
```

```
PS C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\TP> docker cp hadoop-master:/home/livrables .\livrables
Successfully copied 761kB to C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\TP\livrables
PS C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\TP> docker cp hadoop-master:/home/resultat.pdf .\livrables\
Successfully copied 15.9kB (transferred 17.9kB) to C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\TP\livrables\
PS C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\TP> docker cp hadoop-master:/home/top10.txt .\livrables\
Successfully copied 200B (transferred 2.05kB) to C:\Users\franc\Desktop\Maman\Reconversion pro 2026\Diginamic Formation 2026\21 Hadoop\TP\livrables\
```

Figure 18 - Livrables récupérés sur le poste de travail

VI Hive

6.1 Préparation du TSV

Le fichier TSV sans en-tête conserve uniquement les colonnes utiles ville, année et quantité.

Script 'prepare-ventes-tsv.py'

prepare-ventes-tsv.py 1

TP > VI_Hive_Q4 > prepare-ventes-tsv.py > ...

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
# prepare-ventes-tsv : lit le CSV dataw_fro03.csv (champs quotes pouvant contenir
# des virgules) et produit un TSV (uniquement champs 'ville'(villecli),
# 'annee'(datcde), 'quantite'(qte) separes par des tabulations) que
# Hive lit sans ambiguete avec FIELDS TERMINATED BY '\t'.
# Compatible Python 3.5+ (le cluster tourne en 3.11).
#
# Usage :
#   python3 prepare-ventes-tsv.py dataw_fro03.csv ventes-clean.tsv
# ou en flux (stdin -> stdout) :
#   cat dataw_fro03.csv | python3 prepare-ventes-tsv.py > ventes-clean.tsv
#
# Le script COMPTE ce qu'il fait et l'affiche sur stderr (lues / ecrites /
# ecartees, et surtout le nombre de champs "qte" non numeriques :
# c'est ce compteur, pas le nombre de colonnes, qui revele une ligne
# corrompue du jeu fourni — 25 colonnes ne garantissent pas 25 champs sains).

import csv
import sys

try:
    from io import StringIO  # Python 3
except ImportError:  # pragma: no cover
    from StringIO import StringIO  # Python 2 (au cas ou)


def main():
    if len(sys.argv) >= 3:
        fin = open(sys.argv[1], 'r', encoding='utf-8', newline='')
        fout = open(sys.argv[2], 'w', encoding='utf-8', newline='')
    else:
        fin = sys.stdin
        fout = sys.stdout

    reader = csv.reader(fin, delimiter=',', quotechar='"')
    writer = csv.writer(fout, delimiter='\t', lineterminator='\n')

    lues = ecrites = ecartees = 0
    annees_absentes = 0

    for row in reader:
        lues += 1
        # lignes vides / mal formees
        if not row or len(row) != 25:
            ecartees += 1
            continue
```

```
48
49          # en-tete : il a bien 25 colonnes, on le garde tel quel (la table Hive
50          # n'utilise pas skip.header.line.count). On ne lui compte pas de defaut.
51          if row[0] == 'codcli':
52              continue
53
54          ville = row[5].strip()
55          date_commande = row[7].strip()
56          qte_str = row[15].strip()
57          annee = date_commande[:4]
58
59          # champs "quantite" non numeriques : la ligne corrompue est ecrite
60          # , mais comptee.
61          try:
62              qte = int(qte_str)
63          except ValueError:
64              ecartees += 1
65
66          if not ville:
67              ecartees += 1
68              continue
69
70          if len(annee) != 4 or not annee.isdigit():
71              # Hive interprete \N comme NULL. La ligne reste disponible pour Q1,
72              # tandis que Q2 pourra l'exclure avec WHERE annee IS NOT NULL.
73              annee = r"\N"
74              annees_absentes += 1
75
76          # Eviter qu'une tabulation ou un retour ligne casse le format TSV.
77          ville = ville.replace("\t", " ").replace("\r", " ").replace("\n", " ")
78
79          writer.writerow([ville, annee, qte])
80          ecrites += 1
81
82      print("lues      : %d" % lues, file=sys.stderr)
83      print("ecrites   : %d" % ecrites, file=sys.stderr)
84      print("ecartees  : %d" % ecartees, file=sys.stderr)
85      print("annees absentes (NULL Hive) : %d" % annees_absentes, file=sys.stderr)
86
87      if fin is not sys.stdin:
88          fin.close()
89      if fout is not sys.stdout:
90          fout.close()
91
92
93  if __name__ == '__main__':
94      main()
```

Commande de génération sur le terminal hadoop-master (après copie du fichier sur le master)

```
python3 prepare-ventes-tsv.py dataw_fro03.csv ventes-clean.tsv
```

```
root@hadoop-master:/home# python3 prepare-ventes-tsv.py dataw_fro03.csv ventes-clean.tsv
lues      : 135278
ecrites   : 135277
ecartees  : 3
annees absentes (NULL Hive) : 2

-rw-r--r-- 1 root root  2757274 Sep 21 21:56 ventes-clean.tsv
```

Figure 19 - Contrôle de la création du fichier clean TSV

```
root@hadoop-master:/home# wc -l ventes-clean.tsv
135277 ventes-clean.tsv
root@hadoop-master:/home# head -5 ventes-clean.tsv
BOURGUEBUS      2004    2
BOURGUEBUS      2004    2
BOURGUEBUS      2004    1
SAINT MALO      2006    1
ALENCON 2004    3
```

Figure 20 - Contrôle des premières lignes du TSV

6.2 Dépôt du TSV dans HDFS

Commandes exécutées dans hadoop-master

```
hdfs dfs -mkdir -p /user/root/input/TP-Q4
hdfs dfs -put -f ventes-clean.tsv /user/root/input/TP-Q4/
```

```
root@hadoop-master:/home# hdfs dfs -ls /user/root/input
Found 3 items
-rw-r--r--   1 root supergroup     156547 2026-09-15 14:39 /user/root/input/Spotify_Most_Streamed_Songs.csv
drwxr-xr-x   - root supergroup          0 2026-09-21 22:22 /user/root/input/TP-Q4
-rw-r--r--   1 root supergroup   26397166 2026-09-18 23:00 /user/root/input/dataw_fro03.csv
root@hadoop-master:/home# hdfs dfs -put -f ventes-clean.tsv /user/root/input/TP-Q4/
root@hadoop-master:/home# hdfs dfs -ls /user/root/input
Found 3 items
-rw-r--r--   1 root supergroup     156547 2026-09-15 14:39 /user/root/input/Spotify_Most_Streamed_Songs.csv
drwxr-xr-x   - root supergroup          0 2026-09-21 22:22 /user/root/input/TP-Q4
-rw-r--r--   1 root supergroup   26397166 2026-09-18 23:00 /user/root/input/dataw_fro03.csv
root@hadoop-master:/home# hdfs dfs -ls /user/root/input/TP-Q4
Found 1 items
-rw-r--r--   1 root supergroup    2757274 2026-09-21 22:22 /user/root/input/TP-Q4/ventes-clean.tsv
```

Figure 21 - TSV présent dans le répertoire HDFS dédié

6.3 Création de la base et de la table externe

Commandes exécutées dans hadoop-master

```
beeline -u jdbc:hive2://localhost:10000
```

Connection à Hive **:**

```
Connecting to jdbc:hive2://localhost:10000
Connected to: Apache Hive (version 4.1.0)
Driver: Hive JDBC (version 4.1.0)
Transaction isolation: TRANSACTION_REPEATABLE_READ
Beeline version 4.1.0 by Apache Hive
0: jdbc:hive2://localhost:10000>
```

Définition Hive

```
0: jdbc:hive2://localhost:10000> CREATE DATABASE ventes_db;
No rows affected (1.994 seconds)
0: jdbc:hive2://localhost:10000> USE ventes_db;
No rows affected (0.081 seconds)
0: jdbc:hive2://localhost:10000> CREATE EXTERNAL TABLE ventes (
. . . . . .> ville STRING,
. . . . . .> annee INT,
. . . . . .> qte INT
. . . . . .> )
. . . . . .> ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'
. . . . . .> LOCATION '/user/root/input/TP-Q4';
```

```
INFO  : Completed executing command(queryId=root_202
No rows affected (0.356 seconds)
0: jdbc:hive2://localhost:10000> DESCRIBE ventes;
```

Une table externe est créée afin que les données soient persistées sur HDFS (en cas de DROP TABLE).

LOCATION pointe bien vers un dossier HDFS.

```
0: jdbc:hive2://localhost:10000> DESCRIBE ventes;
+-----------+------------+----------+
| col_name  | data_type  | comment  |
+-----------+------------+----------+
| ville     | string     |          |
| annee     | int        |          |
| qte       | int        |          |
+-----------+------------+----------+
3 rows selected (0.513 seconds)

0: jdbc:hive2://localhost:10000> SELECT * FROM ventes LIMIT 5;
+--------------+--------------+-------------+
| ventes.ville | ventes.annee | ventes.qte  |
+--------------+--------------+-------------+
| BOURGUEBUS   | 2004         | 2           |
| BOURGUEBUS   | 2004         | 2           |
| BOURGUEBUS   | 2004         | 1           |
| SAINT MALO   | 2006         | 1           |
| ALENCON      | 2004         | 3           |
+--------------+--------------+-------------+
5 rows selected (1.959 seconds)

0: jdbc:hive2://localhost:10000> SELECT COUNT(*) FROM ventes;
+---------+
|   _c0   |
+---------+
| 135277  |
+---------+
1 row selected (47.971 seconds)
```

Figure 22 - Description et aperçu de la table Hive

6.4 Requête reproduisant Q1

Requête Hive Q1

```
  SELECT ville, SUM(qte) AS total_articles FROM ventes GROUP BY ville ORDER BY total_articles DESC
LIMIT 10;
```

Résultat

```
0: jdbc:hive2://localhost:10000> SELECT ville, SUM(qte) AS total_articles FROM ventes GROUP BY ville ORDER BY total_articles DESC
LIMIT 10;
INFO  : Completed executing command(queryId=root_2026092122
+------------------------+-----------------+
|         ville          | total_articles  |
+------------------------+-----------------+
| LE MANS                | 3019            |
| CAEN                   | 2056            |
| FLERS                  | 1480            |
| LAVAL                  | 1423            |
| VIRE NORMANDIE         | 1405            |
| CHERBOURG EN COTENTIN  | 1362            |
| ALENCON                | 1166            |
| ARGENTAN               | 938             |
| RENNES                 | 874             |
| ATHIS VAL DE ROUVRE    | 809             |
+------------------------+-----------------+
10 rows selected (75.64 seconds)
```

6.5 Requête reproduisant Q2

Requête Hive Q2

```
SELECT CONCAT(ville, '#', CAST(annee AS STRING)) AS ville_annee, SUM(qte) AS total_articles FROM ventes WHERE annee IS NOT NULL GROUP BY ville, annee ORDER BY total_articles DESC LIMIT 10;
```

Résultat

```
0: jdbc:hive2://localhost:10000> SELECT CONCAT(ville, '#', CAST(annee AS STRING)) AS ville_annee, SUM(qte) AS total_articles FROM
ventes WHERE annee IS NOT NULL GROUP BY ville, annee ORDER BY total_articles DESC LIMIT 10;

INFO  : Completed executing command(queryId=root_20260
+--------------------------+-----------------+
|       ville_annee        | total_articles  |
+--------------------------+-----------------+
| LE MANS#2010             | 305             |
| BEAUFORT EN VALLEE#2012  | 305             |
| LE MANS#2006             | 267             |
| LE MANS#2011             | 262             |
| LE MANS#2015             | 258             |
| LE MANS#2008             | 250             |
| LE MANS#2007             | 196             |
| CAEN#2008                | 195             |
| CAEN#2010                | 185             |
| LE MANS#2009             | 185             |
+--------------------------+-----------------+
10 rows selected (76.554 seconds)
```

6.6 Validation croisée avec MapReduce

Les résultats obtenus avec HIVE ou MapReduce sont les mêmes.

6.7 Analyse du plan avec EXPLAIN

Commande EXPLAIN

```
EXPLAIN SELECT ville, SUM(qte) AS total_articles
FROM ventes GROUP BY ville ORDER BY total_articles DESC LIMIT10;
```

```
+----------------------------------------------------+
|                      Explain                       |
+----------------------------------------------------+
| STAGE DEPENDENCIES:                                |
|   Stage-1 is a root stage                          |
|   Stage-2 depends on stages: Stage-1               |
|   Stage-0 depends on stages: Stage-2               |
|                                                    |
| STAGE PLANS:                                       |
|   Stage: Stage-1                                   |
|     Map Reduce                                     |
|       Map Operator Tree:                           |
|           TableScan                                |
|             alias: ventes                          |
|             Statistics: Num rows: 93151 Data size: 15112756 Basic stats: COMPLETE Column stats: NONE |
|             Select Operator                        |
|               expressions: ville (type: string), qte (type: int) |
|               outputColumnNames: ville, qte        |
|               Statistics: Num rows: 93151 Data size: 15112756 Basic stats: COMPLETE Column stats: NONE |
|               Group By Operator                    |
|                 aggregations: sum(qte)             |
|                 keys: ville (type: string)         |
|                 minReductionHashAggr: 0.99         |
|                 mode: hash                         |
|                 outputColumnNames: _col0, _col1    |
|                 Statistics: Num rows: 93151 Data size: 15112756 Basic stats: COMPLETE Column stats: NONE |
|                 Reduce Output Operator             |
|                   key expressions: _col0 (type: string) |
|                   null sort order: z               |
|                   sort order: +                    |
|                   Map-reduce partition columns: _col0 (type: string) |
|                   Statistics: Num rows: 93151 Data size: 15112756 Basic stats: COMPLETE Column stats: NONE |
|                   value expressions: _col1 (type: bigint) |
|       Execution mode: vectorized                   |
|       Reduce Operator Tree:                        |
|         Group By Operator                          |
|           aggregations: sum(VALUE._col0)           |
|           keys: KEY._col0 (type: string)           |
|           mode: mergepartial                       |
|           outputColumnNames: _col0, _col1          |
|           Statistics: Num rows: 46575 Data size: 7556296 Basic stats: COMPLETE Column stats: NONE |
|           File Output Operator                     |
|             compressed: false                      |
```

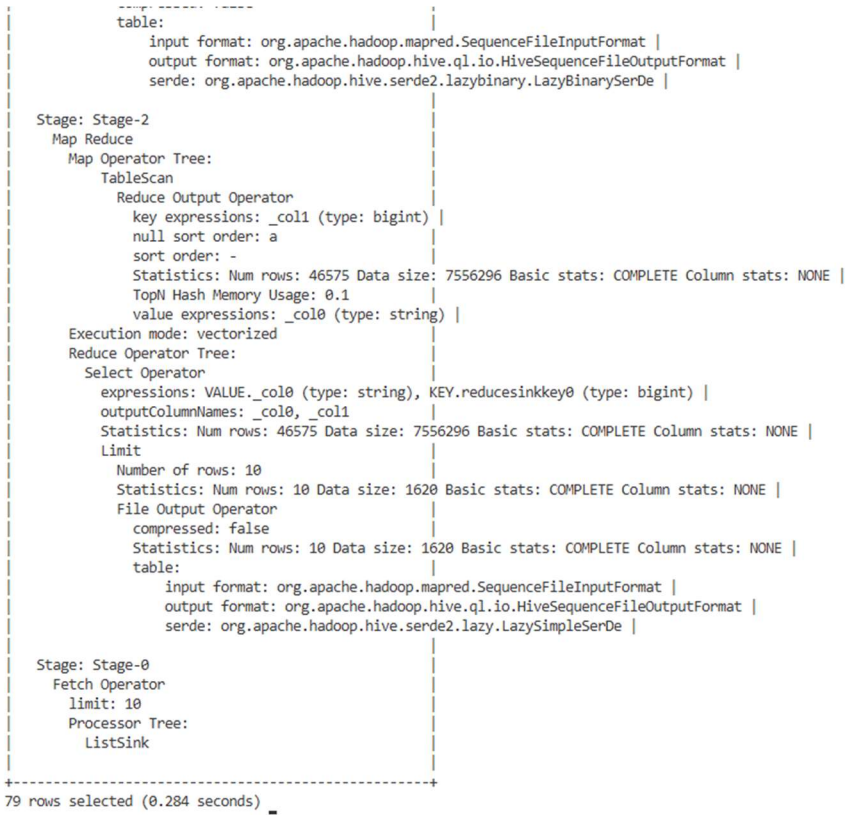


Figure 23 - Plan d'exécution Hive

Cette commande EXPLAIN permet de monter que HIVE traduit une requête SQL en un plan d'exécution distribué s'appuyant sur le moteur MapReduce : lecture de la table externe dans HDFS (TableScan), sélection des colonnes (Select Operator), regroupement par ville et quantité (Group By Operator), transfert des résultats intermédiaires vers les reducers (Reduce Output Operator), écriture du résultat (File Output Operator).

VIII Synthèse et validation croisée

8.1 Cohérence des résultats

Contrôle	Source 1	Source 2	Résultat	Statut
Totaux par ville	MapReduce Q1	Hive Q1	Identique	Conforme~~/ Non conforme~~
Totaux ville et année	MapReduce Q2	Hive Q2	Identique	Conforme~~/ Non conforme~~
Top 10	HBase	Matplotlib et top10.txt	Identique	Conforme~~/ Non conforme~~

8.2 Matrice de contrôle des composants

Fonction	Composant	Rôle dans le TP
Ingère	NiFi	Lit le mini CSV, le transforme et produit le TSV (non traité)
Déplace	Commandes HDFS	Transfère les fichiers entre le conteneur et HDFS
Stocke	HDFS et HBase	Conserve les fichiers sources, sorties et agrégats
Calcule	MapReduce et Hive	Agrège les quantités par ville et par année
Sert	HBase Thrift et Hive	Expose les données aux scripts et aux requêtes

8.3 Conclusion

Ce TP a permis de mettre en œuvre une chaîne Big Data complète, depuis le stockage des données dans HDFS jusqu'à leur traitement avec MapReduce, leur stockage dans HBase et leur analyse avec Hive. Les résultats obtenus par les différentes technologies ont été comparés afin de vérifier leur cohérence. La visualisation du classement des villes a ensuite facilité l'interprétation des données. Ce travail illustre ainsi la complémentarité des composants de l'écosystème Hadoop pour traiter, stocker et analyser des volumes importants de données.